Ecole polytechnique de Louvain

LINFO2146 - Mobile and Embedded Computing

Greenhouse : Improved Project

Academic year 2023-2024

Submission date : August 14th, 2024

Pierre Bastenier (36111700)

Professor:
Ramin Sadre

UCLouvain

Contents

1 Introduction

This document presents the improvements made to the greenhouse project based on initial feedback. The original report, which is included as an annex, details the project's initial state and the issues encountered. The current report addresses these issues by incorporating feedback and implementing necessary enhancements. For a comprehensive understanding of the project's development, the previous report is available in the annex.

2 First Submission Feedback

In order to improve the initial project, it is important to address the feedback provided to enhance the system's performance and functionality. The following points outline the key areas for improvement based on the initial submission:

- **Mobile Terminal Handling:** The initial protocol failed to accommodate the mobile terminal use-case, which was a major requirement. Improvements are needed to ensure that mobile terminals can interact effectively with the system.
- **Multicast Capability:** The protocol did not support multiple greenhouses or multicast operations within the irrigation system. Implementing multicast support is necessary to meet project requirements.
- **Static Gateway Address:** A static address was used for the Gateway without sufficient justification. It is essential to provide a rationale for this choice and consider alternative addressing schemes if necessary.
- **Network Operation Details:** The initial report lacked clarity on the network's operation, such as failure detection mechanisms and the process by which sensors select their parent nodes. Detailed explanations are required to illustrate these aspects comprehensively.

3 Static Address Assignment for Gateway

In a fixed network where the Gateway and Subgateways are assumed to be static and not subject to dynamic joining or leaving, assigning a static address to the Gateway simplifies address management and reduces communication overhead. This approach ensures that all devices have a predictable and consistent endpoint to communicate with, streamlining interactions and avoiding the complexities of dynamic address assignment. Consequently, it enhances the reliability and efficiency of the network, as devices can always address the Gateway in a uniform and straightforward manner.

4 Improvements from Old to New Gateway

The transition from the old gateway implementation to the new one brings several notable improvements :

4.1 Enhanced Subgateway Management

The new gateway code introduces a more sophisticated mechanism for managing subgateways. It supports up to four (randomly chosen, could be more) subgateways, storing their details in an array with a 'subgateway_t' structure that includes the subgateway's address, greenhouse ID, and the timestamp of the last heartbeat. This enhancement allows the gateway to keep track of subgateway connections more effectively.

```
typedef struct {
    linkaddr_t address;
    uint8_t greenhouse_id;
    clock_time_t last_heartbeat;
} subgateway_t;
```

Note that last_heartbeat field is useful for the heartbeat mechanism for managing subgateway status that is included in the code but commented out, as it was not required in the project statements.
As we can observe here, the gateway allows multiple greenhouses (subgateways in yellow) until its maxmium is reached!

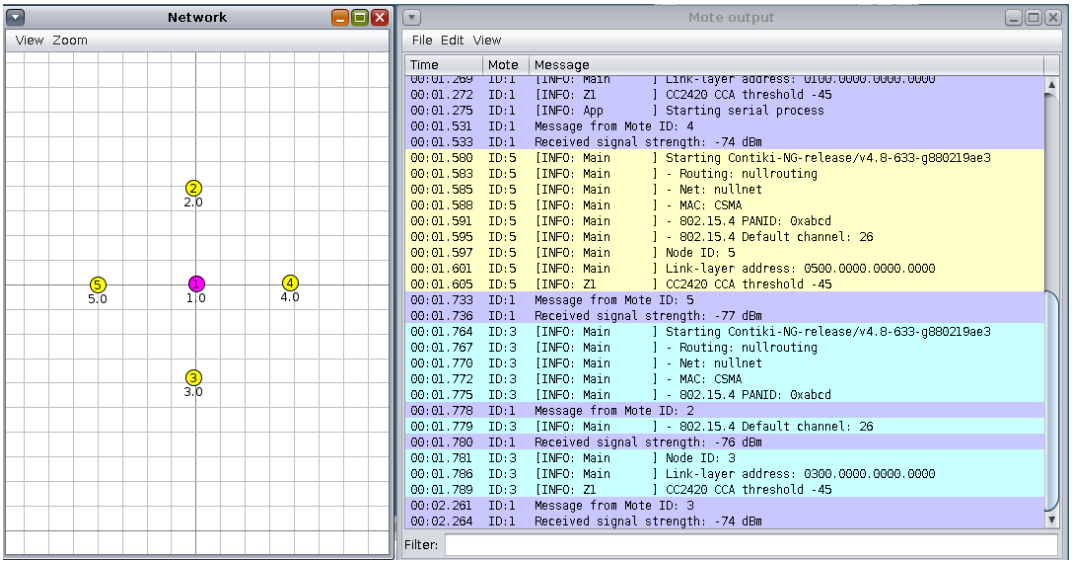

Figure 1: Multiple greenhouses connected to the gateway.

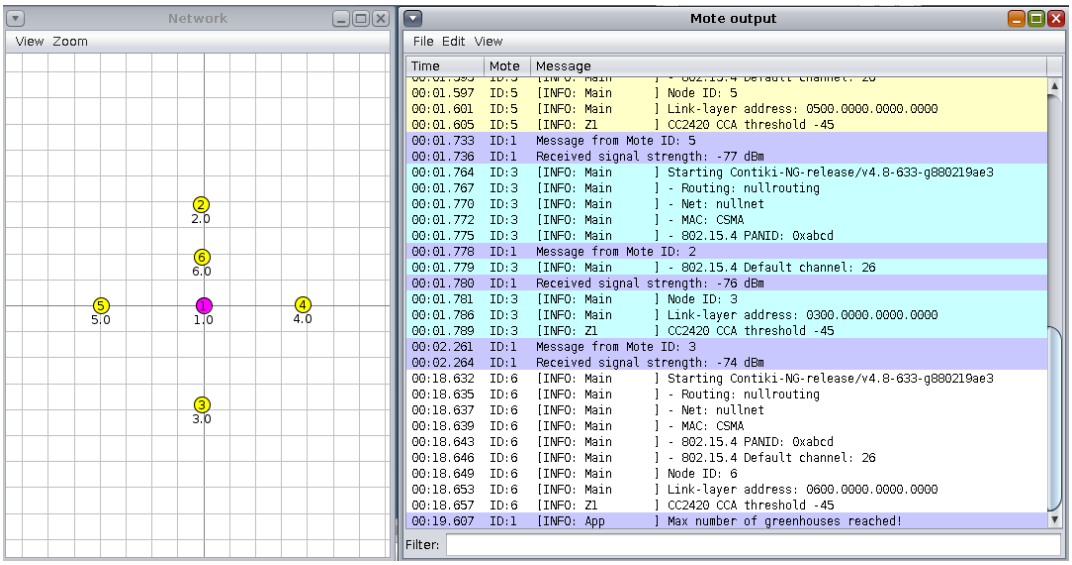

Figure 2: Maximum greenhouses connected reached, node 6 won't connect.

4.2 Multicast Capabilities

The new gateway supports multicast messaging for irrigation and bulb control, allowing commands to be sent to multiple subgateways simultaneously. This is a significant improvement over the old implementation, which used broadcast messaging without the capability to target specific subgateways efficiently.

5 Improvements in the Subgateway Code

In the updated subgateway implementation, several significant improvements have been made compared to the old code. These enhancements include a more robust routing protocol, improved device management, and the introduction of a heartbeat mechanism. The following sections outline these improvements:

5.1 Enhanced Routing Protocol

The new subgateway code introduces a more sophisticated routing protocol with the following features:

- **Dynamic Device Management:** The new routing protocol includes a `routing_table_t` structure that manages up to `MAX_DEVICES` devices. Each device entry now tracks the last heartbeat timestamp, allowing for more efficient monitoring and management of devices.

```
typedef struct {
    device_entry_t devices[MAX_DEVICES];
    int device_count;
    linkaddr_t router; // Router address
} routing_table_t;
```

Listing 1: Definition of the `routing_table_t` structure

- **Device Type Handling:** The updated code distinguishes between different device types (e.g., light sensors, bulbs, irrigation sensors) and manages them accordingly. The `device_entry_t` structure includes a `device_type` field to facilitate this differentiation.

```c
typedef struct {
    linkaddr_t address;
    uint8_t device_type; // e.g., LIGHT_SENSOR 3, BULB 4, IRRIGATION 5
    clock_time_t last_heartbeat; // Timestamp of the last heartbeat
} device_entry_t;
```

Listing 2: Definition of the `device_entry_t` structure

- **Heartbeat Mechanism:** The new implementation includes a heartbeat mechanism between every sensor nodes and subgateways that periodically checks for timed-out devices. Devices that do not send a heartbeat within a specified interval are removed from the routing table. This feature ensures the routing table remains accurate and up-to-date. The heartbeat `CHECK_INTERVAL` allows devices connected to the subgateway to disconnect for a short period of time without being removed, then still communicate with it if it reconnects within the time frame.

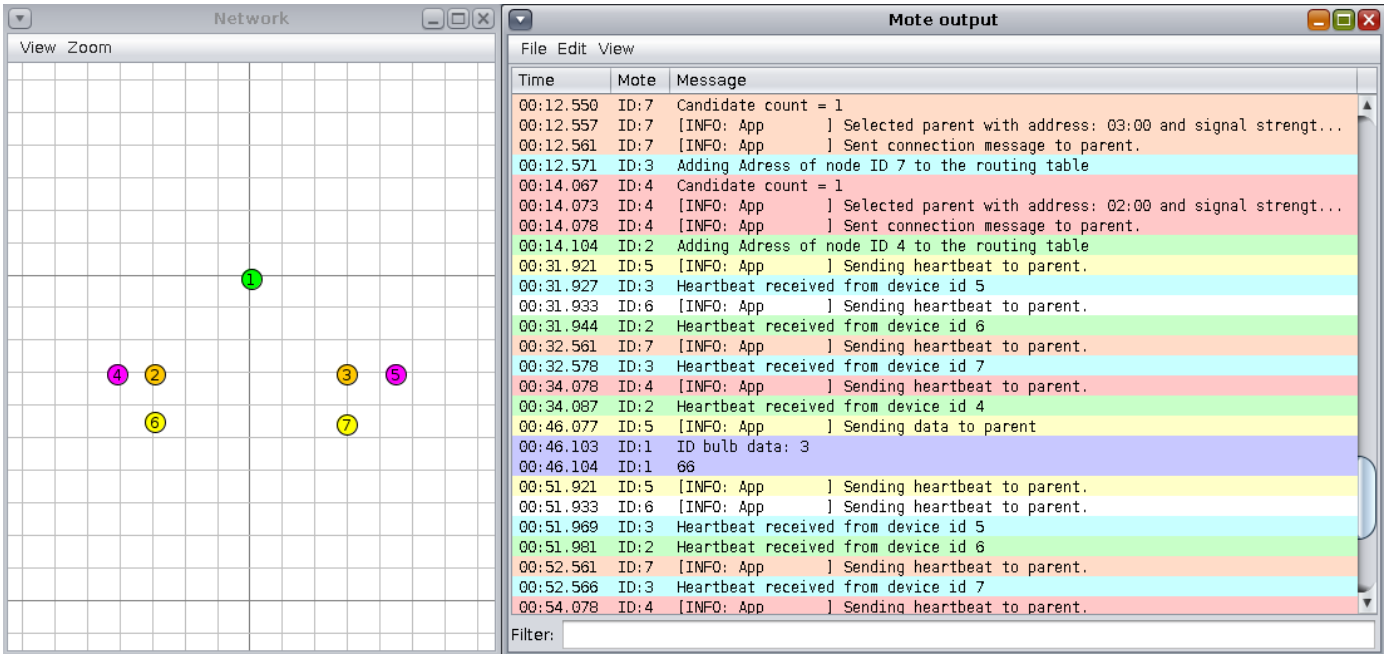

Figure 3: Heartbeat mechanism from devices (yellow : Bulb and purple : Light) connected to the subgateway (node 2 and 3).

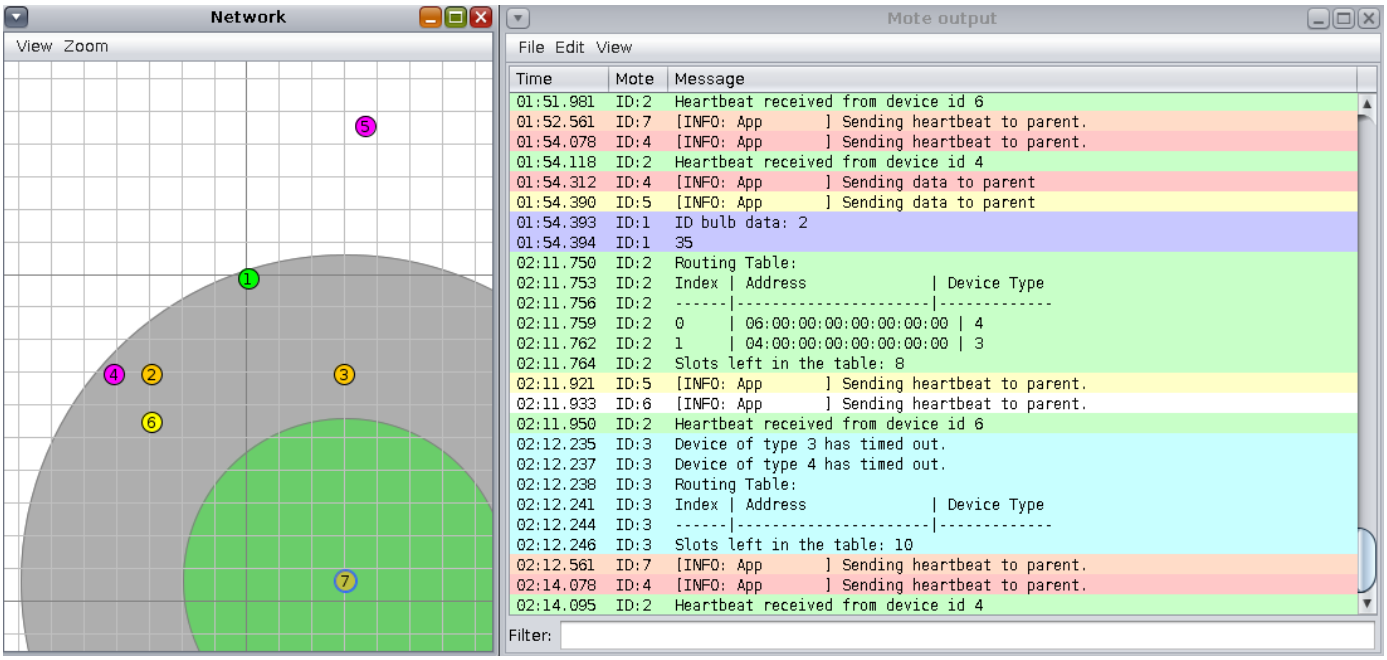

Figure 4: Heartbeat mechanism rises disconnection of device 5 and 7 out of range of the subgateway node 3.

- **Routing Table Printout:** A utility function `print_routing_table()` has been added to output the current state of the routing table, including the number of available slots. This provides better visibility into the routing table's status.

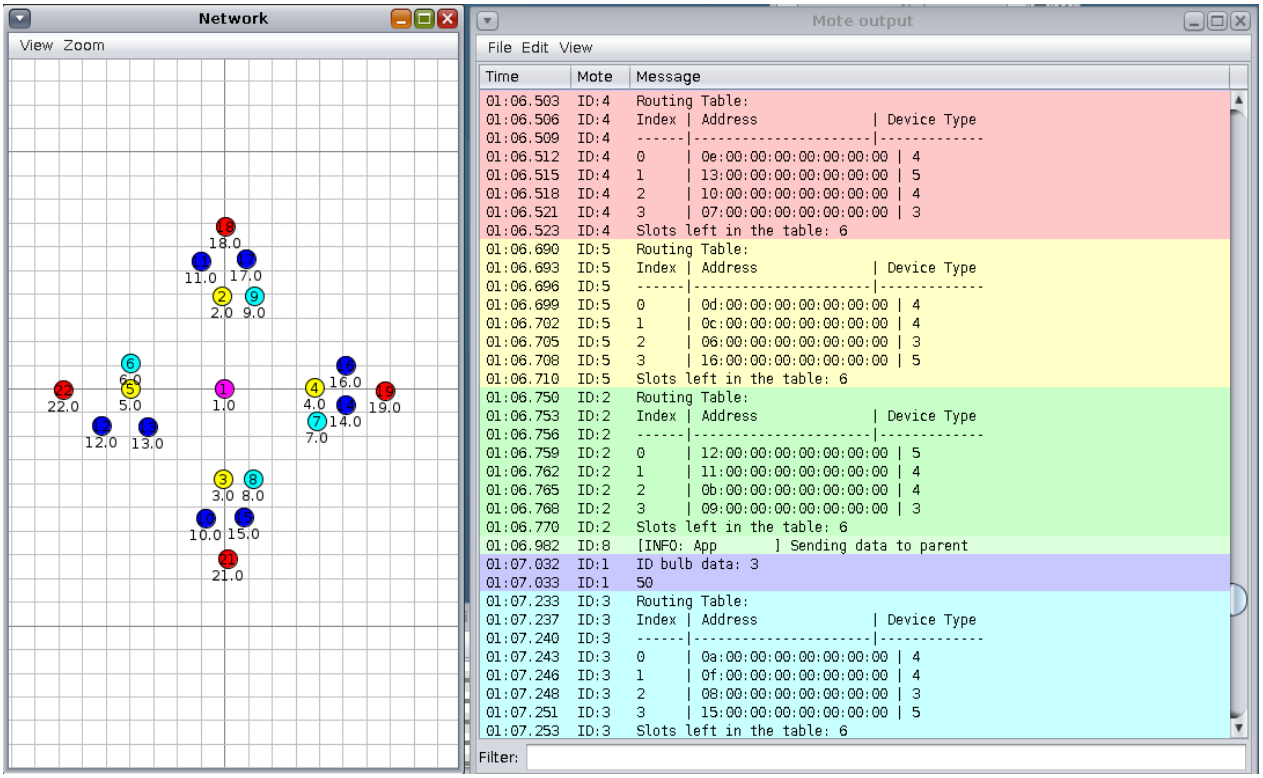

Figure 5: Routing tables of subgateways for a full topology.

- **Improved Device Addition and Removal:** The new implementation efficiently handles the addition of new devices by checking if the device type can be accepted and if there are available slots. It also manages device removal more gracefully, ensuring the routing table remains organized. Note that we suppose there are at most 4 devices connected to the subgateway (1 light, 2 bulbs, 1 irrigation node).
- **Logging and Debugging:** The new implementation includes logging statements (e.g., `LOG_INFO()`) to provide real-time feedback on device interactions and system status. This facilitates debugging and monitoring of the subgateway operation.
- **Routing Table Management Functions:** Several new utility functions enhance the routing protocol, including `address_in_routing_table()`, `find_device_in_routing_table()`, `find_empty_slot()`, and `can_accept_device_of_type()`. These functions streamline the process of managing devices within the routing table.

6 Parent Selection Process

In the subnetwork (within greenhouses), the process of selecting a parent node is crucial for maintaining network hierarchy and ensuring efficient communication. The mechanism described here applies to all sensor nodes, including light sensors, bulbs, and irrigation nodes, which use the same process for parent selection.

6.1 Discovery Phase

Each sensor node periodically broadcasts discovery messages to identify available parent candidates. During this phase, the nodes collect information from responding devices, which includes their address, type, signal strength (RSSI), and ID. This information is stored in a list of `candidate_t` structures. Each candidate is evaluated based on its type, then its signal strength to determine its suitability as a parent.

```
typedef struct {
    linkaddr_t address;
    uint8_t node_type; // Device type
    int16_t signal_strength; // RSSI value
    uint8_t node_id;
} candidate_t;
```

Listing 3: Definition of the `candidate_t` structure

6.2 Backoff Timer

To avoid collisions during communication, a backoff timer is employed. This timer introduces a random delay before a node sends a discovery or connection message. The random backoff time is set to a value between 0 and a maximum defined constant (e.g., 5 seconds). This approach helps to reduce the likelihood of simultaneous transmissions, thereby minimizing message collisions and improving overall network performance.

6.3 Parent Selection

After the discovery phase, nodes evaluate the collected candidates to select the most suitable parent. The selection criteria prioritize nodes based on their type and signal strength. For instance, if a subgateway (node type 2) is available, it is preferred due to its higher capability. If no suitable subgateway is found, any node with the strongest signal is chosen.

The selected parent is then notified, and the node begins periodic communication with it. This includes sending heartbeat messages and data at regular intervals (for the light sensor for example that sends data of the light level). The parent node selection process ensures that each sensor node is efficiently integrated into the network with minimal communication conflicts.

6.4 Illustration Candidate

The following 3 figures show a simple configuration of one gateway (node 1 green), 3 subgateways (2,3,4 yellow) and Light sensors (purple). We will firstly connect 2 Light nodes amongst 3 subgateway that are available. Then once these 2 are connected, we will place a third light node close to the subgateway that are taken. And finally once all 3 are connected, we will try to place a 4th Light node. We observe the expected result, no more slot available for this type of node, but the bulb or irrigation type can still connect to the subgateway using the same process.

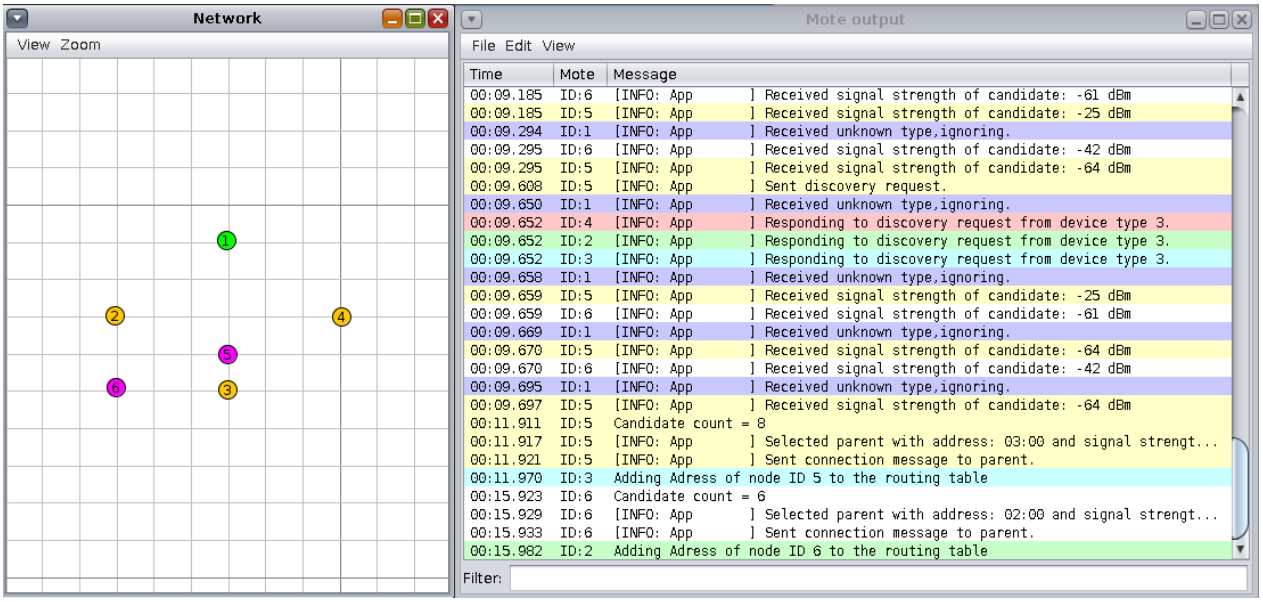

Figure 6: Discovery broadcasted getting candidate, connected to selected parent.

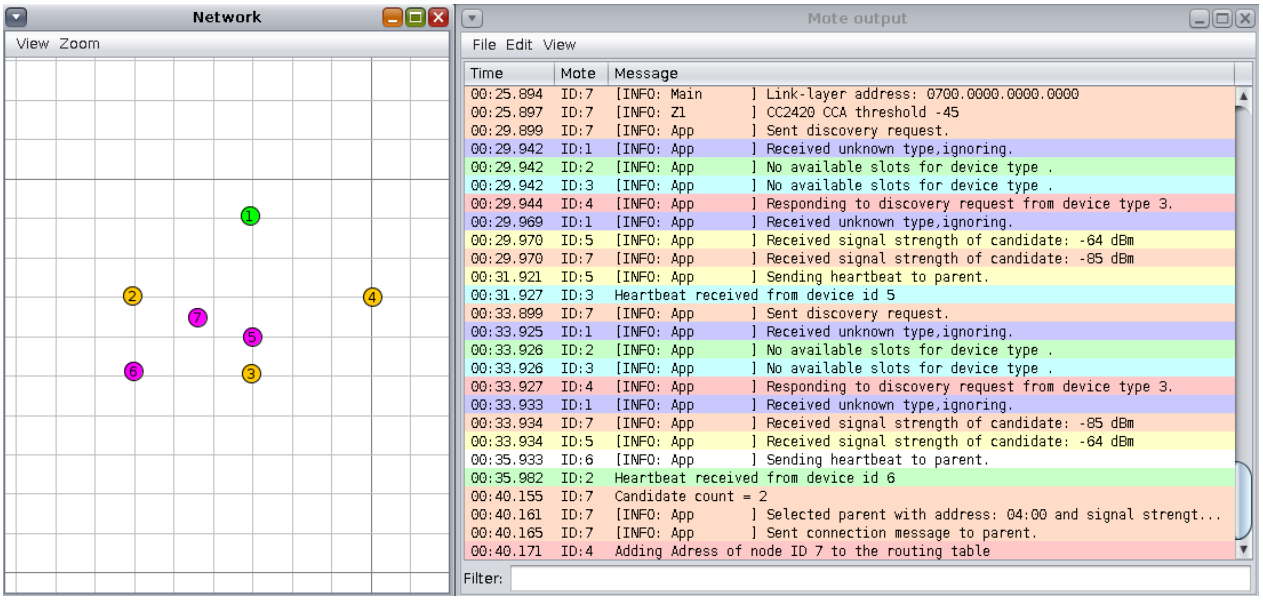

Figure 7: Third Light node launched.

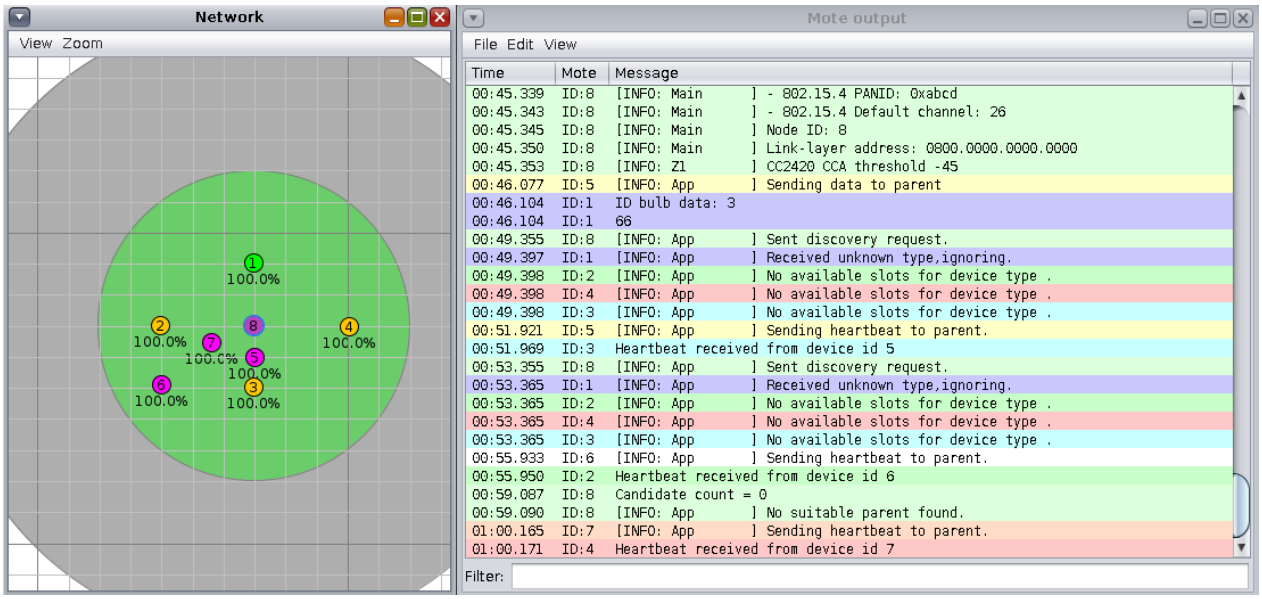

Figure 8: No more slots available for this type of node.

7 Mobile Terminal

The Mobile Terminal has been implemented to connect exclusively with a light sensor node as its parent. It first searches for a light sensor; if none is found, it does not proceed. Although we suppose only one Mobile Terminal should connect to a light sensor at a time, multiple terminals are allowed to be launched simultaneously if each one is assigned to a different greenhouse. Each terminal will connect to the nearest light sensor in its designated greenhouse. If two terminals are equally close to the same sensor, the sensor will communicate with both, sending Z messages to each but it was not specified in the project statement.

After establishing the selection of its parent, the Mobile Terminal sends a connection request and then only exchanges Z maintenance messages before shutting down.

Here is an example of 2 light sensor nodes and 2 mobile terminal. We simplified the network to illustrate clearly what happens.

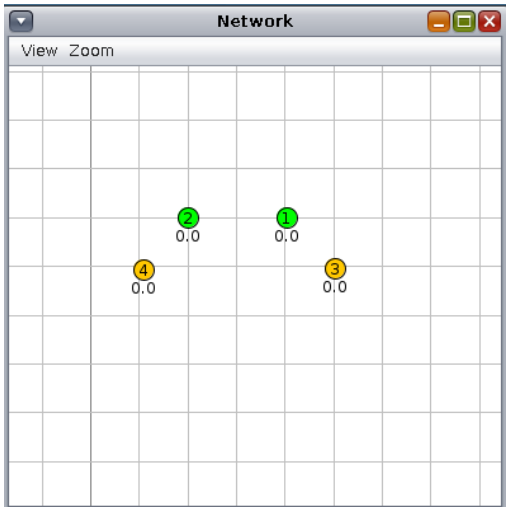

Figure 9: Topology Light sensors (1 and 2) and mobile terminal (3 and 4)

```
00:11.239  ID:4   Candidate count = 7
00:11.246  ID:4   [INFO: App       ] Selected parent with address: 02:00 and signal strength -33 and ID 2
00:11.790  ID:3   Candidate count = 6
00:11.796  ID:3   [INFO: App       ] Selected parent with address: 01:00 and signal strength -34 and ID 1
```

Figure 10: Selection of the parent for the 2 mobiles

```
00:13.250  ID:4   Mobile sent maintenance message 1 to Light sensor ID 2.
00:13.328  ID:4   Mobile received back message number 1 from Light sensor ID 2.
00:13.801  ID:3   Mobile sent maintenance message 1 to Light sensor ID 1.
00:13.833  ID:3   Mobile received back message number 1 from Light sensor ID 1.
00:14.726  ID:1   Candidate count = 0
00:14.730  ID:1   [INFO: App       ] No suitable parent found.
00:15.250  ID:4   Mobile sent maintenance message 2 to Light sensor ID 2.
00:15.328  ID:4   Mobile received back message number 2 from Light sensor ID 2.
00:15.801  ID:3   Mobile sent maintenance message 2 to Light sensor ID 1.
00:15.849  ID:3   Mobile received back message number 2 from Light sensor ID 1.
00:17.250  ID:4   Mobile sent maintenance message 3 to Light sensor ID 2.
00:17.289  ID:4   Mobile received back message number 3 from Light sensor ID 2.
00:17.801  ID:3   Mobile sent maintenance message 3 to Light sensor ID 1.
00:17.857  ID:3   Mobile received back message number 3 from Light sensor ID 1.
00:19.250  ID:4   Mobile sent maintenance message 4 to Light sensor ID 2.
00:19.305  ID:4   Mobile received back message number 4 from Light sensor ID 2.
00:19.801  ID:3   Mobile sent maintenance message 4 to Light sensor ID 1.
00:19.841  ID:3   Mobile received back message number 4 from Light sensor ID 1.
00:21.250  ID:4   Mobile sent maintenance message 5 to Light sensor ID 2.
00:21.328  ID:4   Mobile received back message number 5 from Light sensor ID 2.
00:21.801  ID:3   Mobile sent maintenance message 5 to Light sensor ID 1.
00:21.826  ID:3   Mobile received back message number 5 from Light sensor ID 1.
00:23.250  ID:4   Mobile sent maintenance message 6 to Light sensor ID 2.
00:23.281  ID:4   Mobile received back message number 6 from Light sensor ID 2.
00:23.801  ID:3   Mobile sent maintenance message 6 to Light sensor ID 1.
00:23.854  ID:3   Mobile received back message number 6 from Light sensor ID 1.
00:25.250  ID:4   Mobile sent maintenance message 7 to Light sensor ID 2.
00:25.336  ID:4   Mobile received back message number 7 from Light sensor ID 2.
00:25.801  ID:3   Mobile sent maintenance message 7 to Light sensor ID 1.
00:25.896  ID:3   Mobile received back message number 7 from Light sensor ID 1.
00:27.250  ID:4   Mobile sent maintenance message 8 to Light sensor ID 2.
00:27.289  ID:4   Mobile received back message number 8 from Light sensor ID 2.
00:27.801  ID:3   Mobile sent maintenance message 8 to Light sensor ID 1.
00:27.833  ID:3   Mobile received back message number 8 from Light sensor ID 1.
00:29.250  ID:4   Mobile sent maintenance message 9 to Light sensor ID 2.
00:29.265  ID:4   Mobile received back message number 9 from Light sensor ID 2.
00:29.801  ID:3   Mobile sent maintenance message 9 to Light sensor ID 1.
00:29.849  ID:3   Mobile received back message number 9 from Light sensor ID 1.
00:31.250  ID:4   Mobile sent maintenance message 10 to Light sensor ID 2.
00:31.305  ID:4   Mobile received back message number 10 from Light sensor ID 2.
00:31.308  ID:4   Sent and received 10 messages. Shutting down...
00:31.801  ID:3   Mobile sent maintenance message 10 to Light sensor ID 1.
00:31.880  ID:3   Mobile received back message number 10 from Light sensor ID 1.
00:31.884  ID:3   Sent and received 10 messages. Shutting down...
```

Figure 11: Z messages sent/received of both mobiles

8 Simulation Setup and Procedure

To simulate a general use case, set up an environment with two greenhouses, each equipped with four types of devices: light sensors, two bulbs, and irrigation devices. Additionally, connect a mobile terminal to one of the greenhouses. Follow these steps to configure the simulation:

1. **Create a New Simulation:** Start by creating a new simulation instance.
2. **Designate the Gateway:** Ensure that the first mote created in the simulation is designated as the gateway, as it will receive the initial address.
3. **Enable Serial Socket Server:** Activate the serial socket server on the gateway node.
4. **Add Additional Nodes:** Incorporate the remaining nodes into the simulation.
5. **Adjust Simulation Speed:** Set the simulation to run at a speed of `'x1'`.
6. **Launch Server:** Before starting the simulation, execute `server.py` in a terminal to establish a connection between the server and gateway.

Once configured, observe that the irrigation systems and bulbs function correctly, validating the simulation setup. Once the bulb value sent from the light sensor is above 50, the server indicates to turn them on, as we can see here the light (node 5) sensor from the greenhouse ID 3 sent '66' so the Bulbs 8 and 9 (yellow) are turned ON for a random period of time. We also notice the irrigation started (for a random period of time before irrigation stops) in all greenhouses (blue irrigation node 10 and 11) and the acknowledgment is received. Lastly, when we add the Mobile Terminal node, we can see that it connected to the Light sensor (node 4 purple) and that it sent/received 10 messages before shutting down.

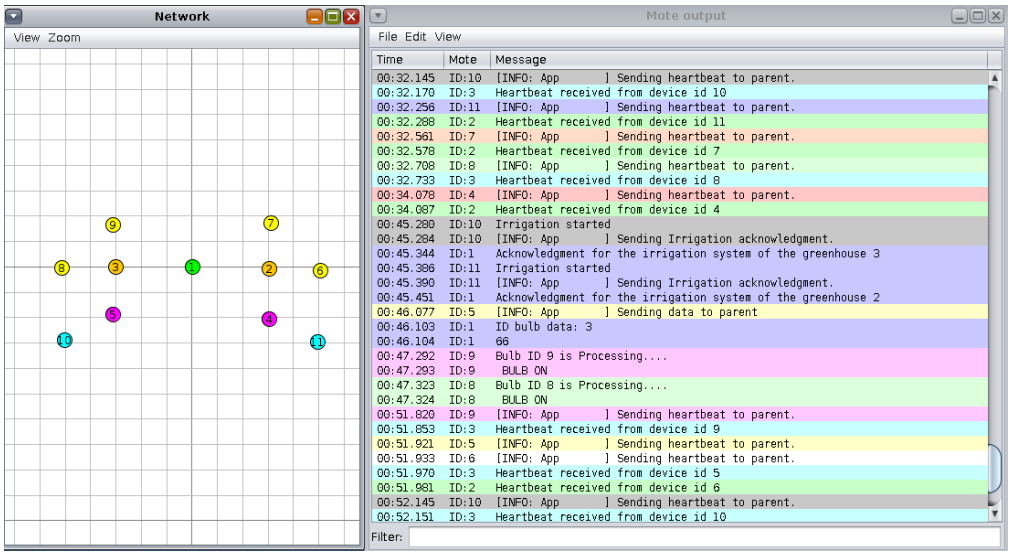

Figure 12: Buld and Irrigation process illustration

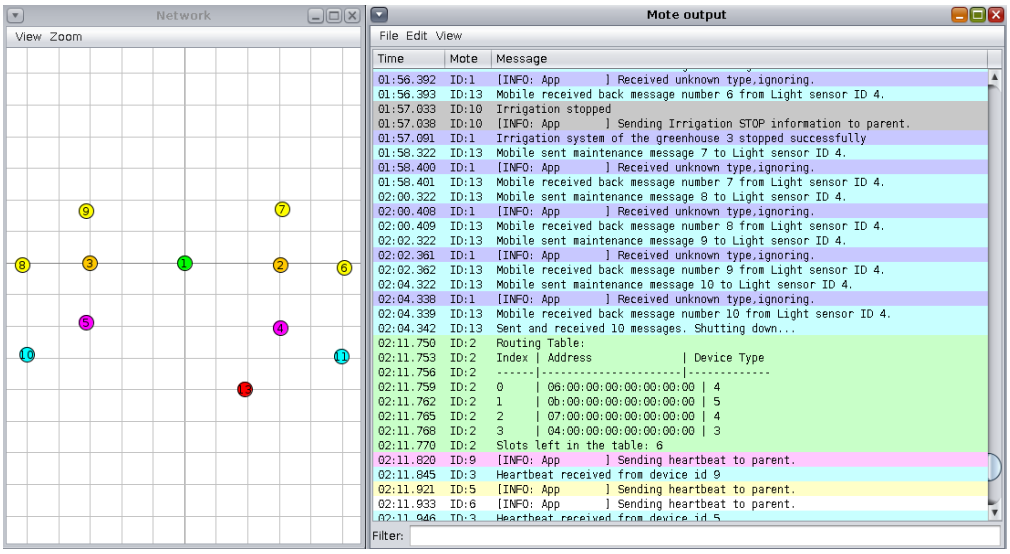

Figure 13: Mobile Terminal added to the configuration.

9 Limitations

Despite significant improvements, several limitations persist due to the constraints and routing protocol of the initial project design:

- **Depth-1 Routing Limitation:** The current system is limited to a depth of 1, where all nodes are directly connected to a subgateway. This restricts the network to simple communication paths and does not support multi-hop routing.
- **Message Structure and Routing:** The existing message structure uses fixed codes (e.g., type, node, id), which is inadequate for multi-hop routing. The protocol does not support downward communication or multi-hop message forwarding.

To enhance further the project and support multi-hop routing, the following modifications are recommended:

- **Redesign Message Structure:** Transition to a dynamic message structure with source and destination addresses, as well as next-hop identifiers. This will allow nodes to route messages based on addresses rather than fixed codes.
- **Enhance Routing Protocol:** Implement a multi-hop routing protocol using dynamic routing tables that include next-hop information. This will enable nodes to forward messages through the network to reach their destination. With our current implementation it would be something like :

```
typedef struct {
    linkaddr_t address;         // Address of the node
    linkaddr_t next_hop;        // Address of the next hop to reach this node
    uint8_t device_type;        // Type of device (e.g., LIGHT_SENSOR, BULB, IRRIGATION)
} device_entry_t;
```

A ANNEX

A.1 Code Structure

1. **Motes:**
    - gateway.c
    - subgateway.c
    - bulb.c
    - light.c
    - irrigation.c
2. **server.py:** Server that sends messages to the gateway and also receives from it.
3. **Makefile**
4. **Helper files:**
    - helper.h: Definition of helper functions and packet structure
    - helper.c: Implementation of the functions

A.2 Motes & Server

A.2.1 Server

The server establishes a TCP connection to a specified IP address and port, attempts to reconnect 10 times if the connection fails, and once connected, continuously reads messages from the gateway. It sends a message every 5 seconds to the gateway to start irrigation for all greenhouses. Additionally, it checks for light sensor messages and sends a message to the gateway to turn on the bulbs if the light level falls below 50.

A.2.2 Gateway

This node implements two processes: one listens to the server and modifies global flags according to the received messages, while the other handles periodic events, checks the flags, and sends broadcast messages for irrigation and bulb activation.

A.2.3 Subgateway

This device handles connections from bulbs, light sensors, and irrigation sensors, acting as a bridge between them and the central gateway.

A.2.4 Irrigation System

This system defines a process called irrigation that sends a connection message to the Subgateway. It waits for a message of type 4 from the server, activates irrigation for 2 seconds, and sends a stop message back to the server.

A.2.5 Light

This node implements the light sensor, which sends a random value between 0 and 100 every two seconds to represent luminosity.

A.2.6 Bulb

Turns on for 5 seconds upon receiving a message from the server.

A.3 Message Format

We implemented the message format in ‘helper.h‘, using various variables:

1. **Type:** Defines the kind of message. Possible types include:
   (a) New connection
   (b) Connection accepted
   (c) Sensor data
   (d) Irrigation
2. **Unicast:** Indicates whether the gateway needs to broadcast the information to all greenhouses.
3. **Id:** Differentiates between greenhouses. Although not used in the current implementation, it allows the server and gateways to identify the destination of the message.
4. **Node:** Identifies the source of the message (gateways, irrigation, etc.), useful for routing purposes.
5. **Signal:** Indicates the quality of the connection. Not used in the current implementation due to the lack of priority connection for motes.
6. **Data:** Carries sensor data and indicates if the bulb needs to be turned on.

A.4 Routing

A.4.1 Routing between Gateway and Subgateway

The Gateway's IP address (0100.0000.0000.0000) is assumed to be known by all Subgateways. This functionality has not been fully implemented yet and will be discussed in the improvements section. Ideally, the Gateway should store the addresses of Subgateways in a ‘linkaddr‘ array and assign specific IDs to each Subgateway. Currently, the Gateway simply broadcasts messages.

A.4.2 Routing between Subgateway, Light Sensor, Bulb, and Irrigation System

The Light Sensor, Bulb, and Irrigation System in a greenhouse first broadcast a type 0 message to announce themselves to the Subgateway. The Subgateway stores each IP address in a ‘linkaddr‘ structure and subsequently unicasts the data received from the server to the Bulb and Irrigation System.

A.5 Running the Simulations and Tests

The server runs on Linux, outside the Cooja simulation but on the same network. To establish a connection between the server and the gateway, retrieve the IP address of the Docker container using:

- `$ docker ps`: Retrieve the Docker container name.
- `$ docker inspect <docker_name>`: Find the IP address (e.g., `"IPAddress" : "172.17.0.2"`).

Enable the Serial Socket Server on the gateway in Cooja to listen on a specific port (e.g., 60001). Launch the server with:

```
python3 server.py --ip 172.17.0.2 --port 60001
```

Create the gateway mote first to get the root address `0100.0000.0000.0000`, enable the Serial Socket Server, create other motes, run the simulation, and finally start the server.

The figure below shows a simulation of one greenhouse, consisting of a Subgateway (mote 2), Light Sensor (mote 3), Bulb (mote 5), and Irrigation System (mote 4) connected to the main Gateway (mote 1).

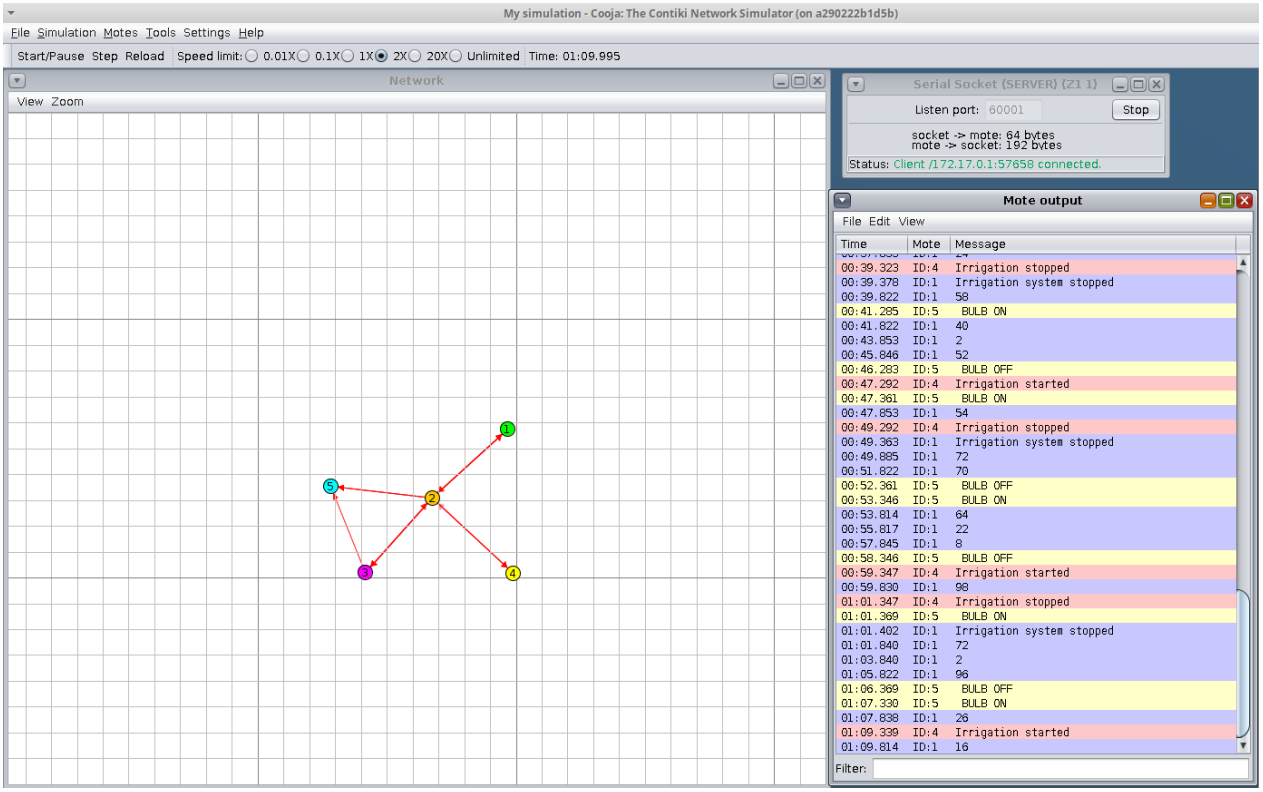

Figure 14: Simulation of one Greenhouse

A.6 Possible Improvements

The current implementation functions correctly but has some limitations. The mobile device was not implemented due to challenges in measuring signal strength between motes. Additionally, the gateway can currently support only one greenhouse. This can be addressed by adding a routing table for Subgateway addresses, similar to the approach used for greenhouse devices. Due to time constraints, this complexity was not incorporated in the current project.

A.7 Conclusion

The project meets essential requirements but has notable weaknesses, including support for only one greenhouse and lack of mobile terminal implementation.

A.8 Github Repository

The entire project with all files related can be found on our GitHub repository.